

AquaWise

Smart Water Conservation and Awareness Portal

Software Engineering Deliverables

Use-case diagram · Use-case specifications · ER diagram · DFD Level 0 and Level 1

Web Programming Laboratory — Semester 5

K J Somaiya

Mapped to UN SDG 6: Clean Water and Sanitation

Team: Jaineel, [teammate name]

Date: 22 September 2026

Contents

1. UML use-case diagram
 2. Use-case specifications (6 key use cases)
 3. ER diagram (MongoDB collections)
 4. Data-flow diagrams: Level 0 and Level 1
 5. Viva explanation of the use-case diagram
- Appendix A. PlantUML source — use-case diagram
- Appendix B. Source and image files

*All diagrams are derived from the AquaWise Requirements Specification (FR, actor and data-model sections).
Identifiers such as FR-C3 or FR-S12 refer to that document.*

1. UML use-case diagram

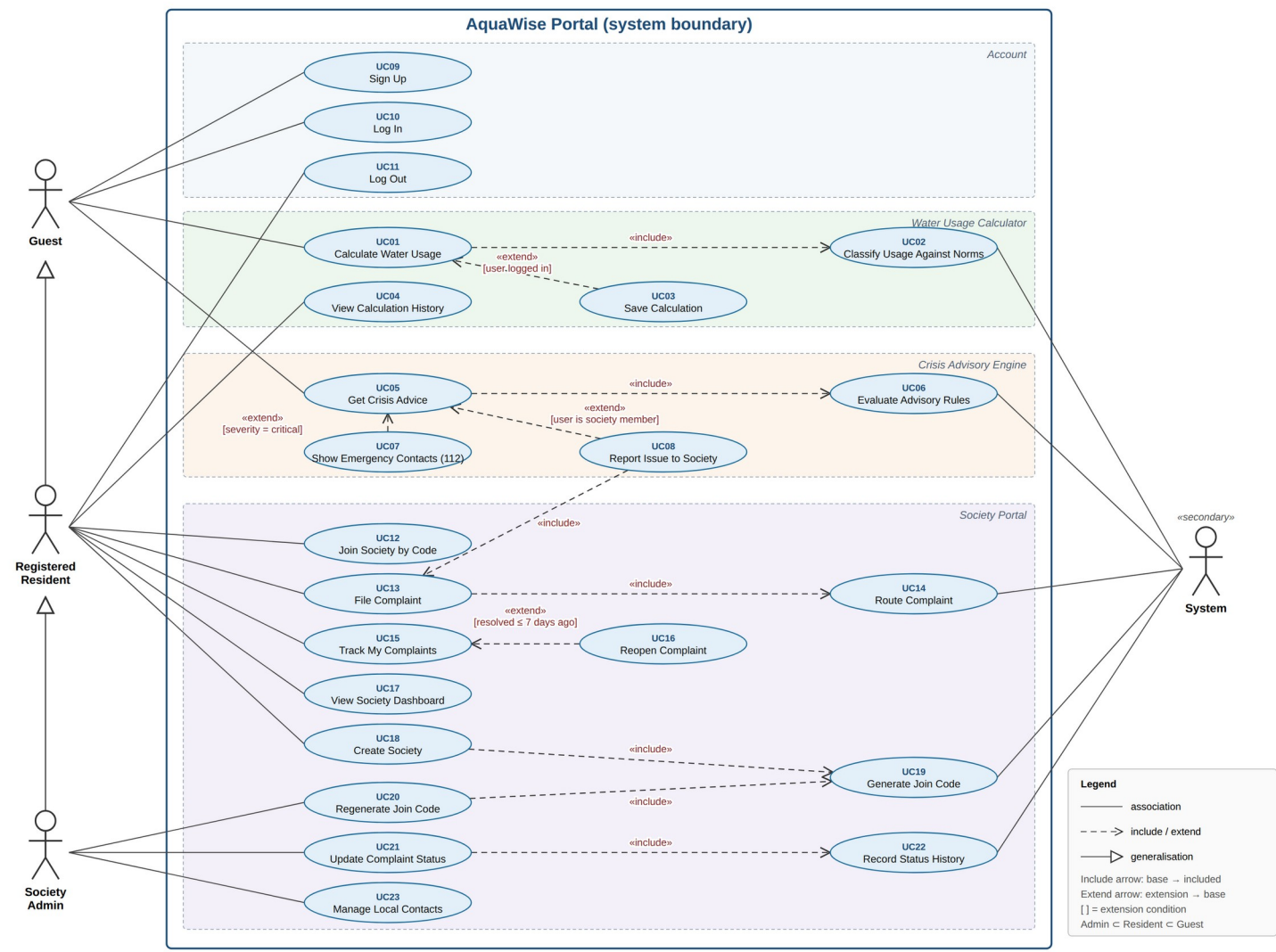


Figure 1. AquaWise use-case diagram — 4 actors, 23 use cases in 4 subsystems, with include, extend and generalisation relationships.

1.1 Actors

Actor	Type	Description
Guest	Primary	Any visitor who is not logged in. Uses the calculator and advisory, and can sign up or log in.
Registered Resident	Primary (specialises Guest)	Logged-in user. Inherits all Guest use cases; can create/join a society, file and track complaints, view the dashboard.
Society Admin	Primary (specialises Resident)	Resident who created the society. Inherits all Resident use cases; updates complaint status, manages contacts, regenerates the join code.
System	Secondary	The AquaWise server acting automatically: classifies usage against norms, evaluates advisory rules, routes complaints, generates join codes, records status history.

Note: the requirements document also defines a System Admin (a seeded demo account). It is left out of this diagram, as specified for this deliverable; its permissions are a superset of Society Admin.

1.2 Use-case catalogue

ID	Use case	Primary actor	Relationships
UC01	Calculate Water Usage	Guest	Includes UC02; extended by UC03
UC02	Classify Usage Against Norms	System	Included by UC01
UC03	Save Calculation	Resident	Extends UC01 [user logged in]
UC04	View Calculation History	Resident	—
UC05	Get Crisis Advice	Guest	Includes UC06; extended by UC07, UC08
UC06	Evaluate Advisory Rules	System	Included by UC05
UC07	Show Emergency Contacts (112)	Guest	Extends UC05 [severity = critical]
UC08	Report Issue to Society	Resident	Extends UC05 [society member]; includes UC13
UC09	Sign Up	Guest	—
UC10	Log In	Guest	—
UC11	Log Out	Resident	—
UC12	Join Society by Code	Resident	—
UC13	File Complaint	Resident	Includes UC14
UC14	Route Complaint	System	Included by UC13
UC15	Track My Complaints	Resident	Extended by UC16
UC16	Reopen Complaint	Resident	Extends UC15 [resolved $\leq$ 7 days ago]
UC17	View Society Dashboard	Resident	—
UC18	Create Society	Resident	Includes UC19
UC19	Generate Join Code	System	Included by UC18, UC20
UC20	Regenerate Join Code	Society Admin	Includes UC19
UC21	Update Complaint Status	Society Admin	Includes UC22
UC22	Record Status History	System	Included by UC21
UC23	Manage Local Contacts	Society Admin	—

2. Use-case specifications

The six use cases below cover all three graded functionalities plus authentication and complaint handling. Step numbers in alternate and exception flows refer to the main flow.

UC01 — Calculate Water Usage

ID	UC01
Name	Calculate Water Usage
Actor	Guest (inherited by Resident and Society Admin); secondary: System
Preconditions	• User is on the Calculator page. No login is required. • Formula constants version is loaded (formulaVersion "1.0").
Main flow	1. User opens the Calculator page. 2. System shows the form pre-filled with default values and their sources (FR-C2). 3. User enters adults, children, city type, bathing mode and shower details, toilet type, washing-machine loads and type, garden area, tap-running minutes, tap type, dripping taps and, optionally, litres received per day. 4. User clicks Calculate. 5. System validates every field against the allowed ranges (FR-C12). 6. System computes litres per activity with the Section 7.2 formulas, the household total and per-capita use in lpcd (FR-C3, FR-C4). 7. System performs UC02 Classify Usage Against Norms: assigns the band and over-use / under-supply flags (FR-C6 to FR-C8). 8. System displays the activity chart and table, band, norm compared against, and the top 3 saving tips with litres saved (FR-C5, FR-C9).
Alternate flows	• A1 (step 3): User keeps the defaults; flow continues at step 4. • A2 (step 3): User enters litres received per day; at step 7 the supply check adds "Under-supply vs JJM norm" or "Supply below estimated need". • A3 (step 8): User is logged in → extension UC03 Save Calculation: server recomputes, stores a CalculatorLog and shows "Saved to history" (FR-C10, FR-C11). • A4 (step 8): User edits any input → flow returns to step 4.
Exception flows	• E1 (step 5): A value is missing, non-numeric or out of range → field-level message; no result shown; focus moves to the first invalid field. • E2 (step 5): Adults = 0 or adults + children > 30 → "Household must have 1–30 members, at least 1 adult". • E3 (A3): Server or database unavailable → result stays on screen (computed on the client); message "Could not save — try again".
Postconditions	• Result with band and flags is displayed. • For logged-in users, one CalculatorLog is stored; nothing is stored for guests.

UC05 — Get Crisis Advice

ID	UC05
Name	Get Crisis Advice
Actor	Guest (inherited by Resident and Society Admin); secondary: System
Preconditions	Advisory rules (14 rules, D5) are loaded on the server.
Main flow	1. User opens the Advisory page. 2. System lists the 5 problem types: leakage, no supply, unsafe drinking water, low pressure, contamination (FR-A1). 3. User selects a problem type. 4. System asks the 1–3 trigger questions for that type (FR-A2). 5. User answers and submits. 6. System performs UC06 Evaluate Advisory Rules: checks rules from most to least severe and picks the first match (FR-A7). 7. System shows the severity, the numbered steps in stored order and the contacts with numbers and when to call (FR-A3, FR-A4).

	8. User optionally prints or copies the steps (FR-A9).
Alternate flows	• A1 (step 7): Severity is critical → extension UC07 Show Emergency Contacts: red banner with 112 (FR-A5). • A2 (step 7): User is a logged-in society member → contacts come from their Society record, and a "Report this to my society" button is shown → extension UC08 opens UC13 File Complaint pre-filled with category, severity and rule ID (FR-A8). • A3 (step 5): User goes back and changes an answer → flow returns to step 6.
Exception flows	• E1 (step 6): No rule matches the answers → system shows the lowest-severity rule for that type plus 112, and logs the case for review. • E2 (step 6): Rules cannot be loaded → "Advisory temporarily unavailable" with the 112 emergency number. • E3 (A2): A Guest clicks "Report" → redirected to UC10 Log In.
Postconditions	• Guidance and contacts are displayed. • Nothing is stored unless the user continues to UC08.

UC10 — Log In

ID	UC10
Name	Log In
Actor	Guest
Preconditions	• User has a registered account with isActive = true. • User is not already logged in.
Main flow	1. User opens the Login page. 2. User enters email and password. 3. User submits the form. 4. System checks both fields are present and the email is well-formed. 5. System finds the user by lower-cased email. 6. System compares the password with the stored bcrypt hash. 7. System regenerates the session ID, stores userId and role in the session (D6) and sets an httpOnly cookie (NFR-S2). 8. System updates lastLoginAt. 9. System redirects: society member → Society Dashboard; no society yet → Join / Create Society page. The navbar shows role-specific links.
Alternate flows	• A1 (step 1): User is already logged in → redirected straight to the dashboard. • A2 (step 9): User was sent to Login from a protected page → returned to that page after login.
Exception flows	• E1 (step 5 or 6): Email not found or password wrong → HTTP 401 with the generic message "Invalid email or password" (does not reveal which). • E2 (step 3): 5 failed attempts from one IP in 15 minutes → HTTP 429 "Too many attempts, try again later" (NFR-S4). • E3 (step 5): isActive = false → HTTP 403 "Account deactivated". • E4 (step 4): Empty or malformed field → client-side validation message; request not sent.
Postconditions	An authenticated session exists for 2 hours; later requests carry the session cookie.

UC12 — Join Society by Code

ID	UC12
Name	Join Society by Code
Actor	Registered Resident
Preconditions	• User is logged in. • User.society is null (not yet in a society).
Main flow	1. User opens the Join Society page. 2. User enters the 6-character join code, flat number and wing. 3. User submits.

	4. System validates the code format (6 characters, A-Z and 2-9) and flat number. 5. System looks up an active society with that joinCode. 6. System shows the society name and area for confirmation. 7. User confirms. 8. System sets user.society, flatNo and wing, and increments society.memberCount. 9. System redirects to the Society Dashboard (UC17).
Alternate flows	• A1 (step 2): Code typed in lower case → system converts it to upper case. • A2 (step 7): User cancels → nothing is saved; user returns to the Join page.
Exception flows	• E1 (step 5): No society has that code, or the code was regenerated → "Invalid or expired join code". • E2 (precondition): User already belongs to a society → HTTP 409 "You are already a member of <society name>". • E3 (step 4): Flat number empty or has invalid characters → field-level error. • E4 (any step): Session expired → redirected to UC10 Log In.
Postconditions	User is linked to exactly one society and can now file complaints and view its dashboard.

UC13 – File Complaint

ID	UC13
Name	File Complaint
Actor	Registered Resident (also Society Admin); secondary: System
Preconditions	User is logged in and is a member of a society.
Main flow	1. User opens New Complaint. 2. User selects category, severity and location, and enters title and description. 3. User submits. 4. System validates all fields (title 5–80, description 10–1000 characters, enum values). 5. System generates the next complaint number, e.g., AQW-2026-0001 (FR-S10). 6. System performs UC14 Route Complaint: picks the first society contact whose categories include the complaint category, and stores a snapshot in assignedContact (FR-S12). 7. System saves the complaint with status "active", society and filedBy taken from the session, and the first statusHistory entry (null → active). 8. System shows the complaint number and the assigned contact's name and phone; dashboard counts update.
Alternate flows	• A1 (step 1): Started from UC08 → category, severity and advisoryRuleId are pre-filled; user may change severity. • A2 (step 6): No contact handles the category → complaint is routed to the society admin.
Exception flows	• E1 (step 4): Validation fails → field-level errors; form content is kept. • E2 (step 3): Request body names a different society → ignored; society is always taken from the session (HTTP 403 if the user has none). • E3 (step 7): Database write fails → HTTP 500 "Could not file complaint — try again"; form content is kept.
Postconditions	• One complaint exists in D3 with status active and a routed handler. • It appears on the society dashboard and in the filer's My Complaints list.

UC21 – Update Complaint Status

ID	UC21
Name	Update Complaint Status
Actor	Society Admin; secondary: System
Preconditions	• User is logged in with role society_admin. • The complaint belongs to the admin's society and is not resolved.
Main flow	1. Admin opens the Society Dashboard and filters the list by status or category. 2. Admin opens a complaint. 3. System shows details, status history and the allowed next statuses.

	4. Admin selects the next status (active → in_progress, or in_progress → resolved) and writes a note. 5. Admin submits. 6. System checks the transition is allowed and the note is not empty. 7. System performs UC22 Record Status History: appends {from, to, by, note, at} to statusHistory (FR-S14). 8. System updates the status; for "resolved" it sets resolvedAt. 9. Dashboard counts refresh; the filer sees the new status in UC15 Track My Complaints.
Alternate flows	• A1 (step 4): Admin resolves an active complaint directly (active → resolved is allowed). • A2 (after step 9): Filer reopens within 7 days (UC16) → status returns to active, resolvedAt is cleared, and it reappears in the admin's active list.
Exception flows	• E1 (step 6): Transition not allowed (e.g., resolved → in_progress) → HTTP 400 "Invalid status change". • E2 (step 2): Complaint belongs to another society → HTTP 403. • E3 (step 6): Note is empty → field-level error. • E4 (step 5): Status changed since the page loaded → HTTP 409 "Complaint was updated — reload".
Postconditions	• Complaint has the new status and one new, unchangeable history entry. • Dashboard counts match the database.

3. ER diagram (MongoDB collections)

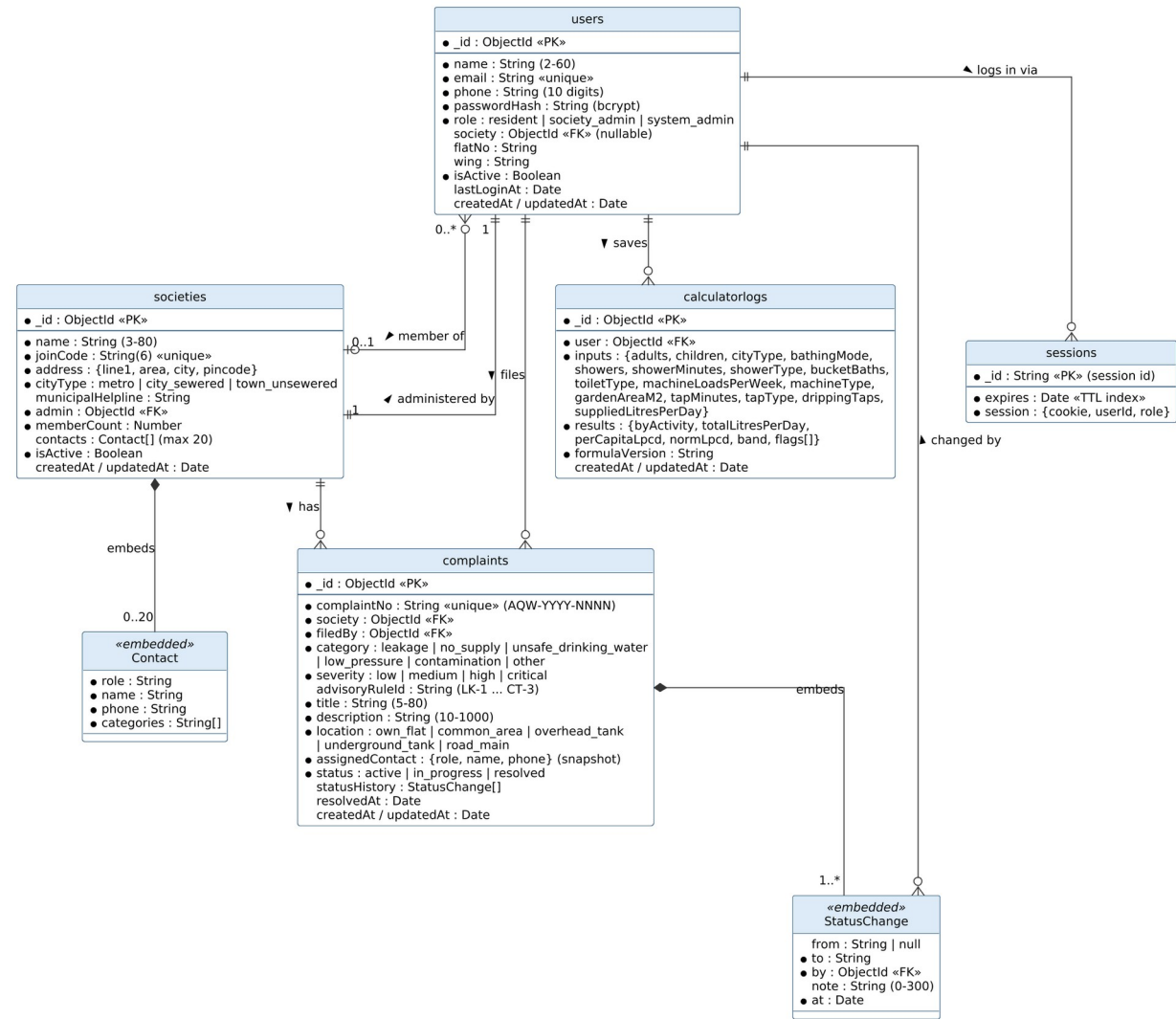


Figure 2. AquaWise ER diagram — 5 collections and 2 embedded sub-documents, crow's-foot notation.

3.1 Relationships

Relationship	Cardinality	How it is stored
User — member of — Society	Many users : 0 or 1 society	users.society (ObjectId ref)
Society — administered by — User	1 : 1	societies.admin (ObjectId ref)
Society — has — Complaint	1 : many	complaints.society (ObjectId ref)
User — files — Complaint	1 : many	complaints.filedBy (ObjectId ref)
User — saves — CalculatorLog	1 : many	calculatorlogs.user (ObjectId ref)
User — logs in via — Session	1 : many	sessions.session.userId (connect-mongo)
Society — embeds — Contact	1 : 0..20	societies.contacts[] (embedded array)
Complaint — embeds — StatusChange	1 : 1..many	complaints.statusHistory[] (embedded array)
StatusChange — changed by — User	many : 1	statusHistory[].by (ObjectId ref)

Design notes

- Contacts and status history are embedded because they are always read with their parent and never queried alone; everything else is referenced so each collection stays small and indexable.
- assignedContact is copied into the complaint at filing time, so editing a society contact later does not rewrite old complaints.
- The sessions collection is created automatically by connect-mongo for express-session; a TTL index on expires removes old sessions. It is shown here because it is a real collection in the database.
- Unique indexes: users.email, societies.joinCode, complaints.complaintNo. Query indexes: complaints {society, status, createdAt}, calculatorlogs {user, createdAt}.

4. Data-flow diagrams

4.1 Level 0 — context diagram

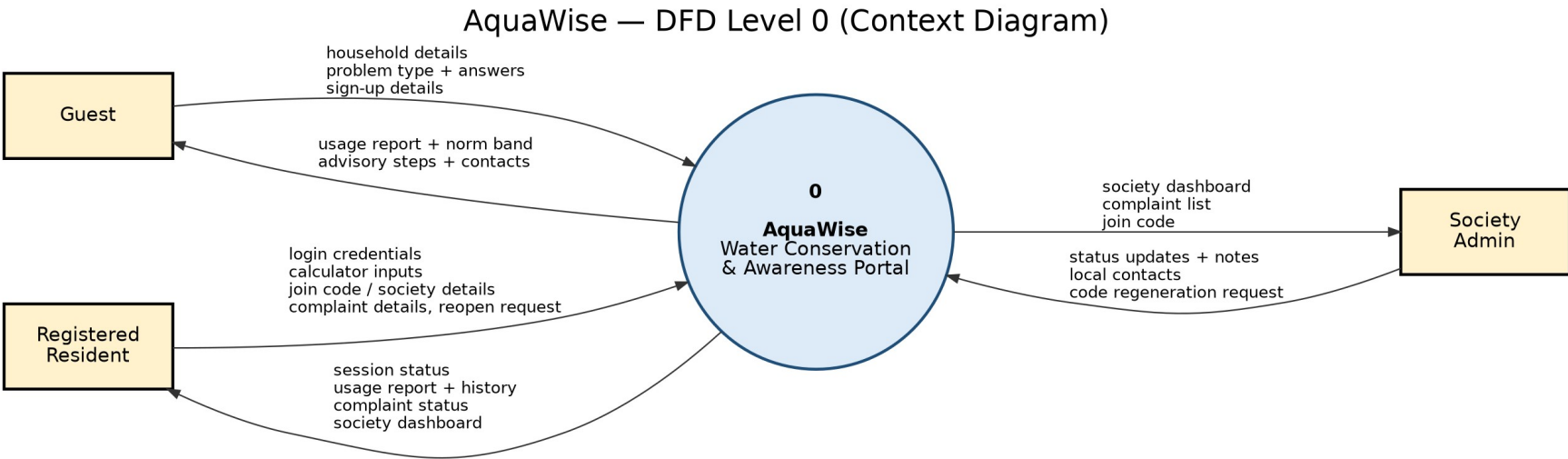


Figure 3. DFD Level 0: the whole portal as one process (0) exchanging data with three external entities.

The context diagram treats AquaWise as a single process. Guests send household details and problem descriptions and receive usage reports and advisory steps. Residents add credentials, society joining and complaints, and receive their history, complaint status and the dashboard. The Society Admin sends status updates and contacts, and receives the complaint list and join code.

4.2 Level 1

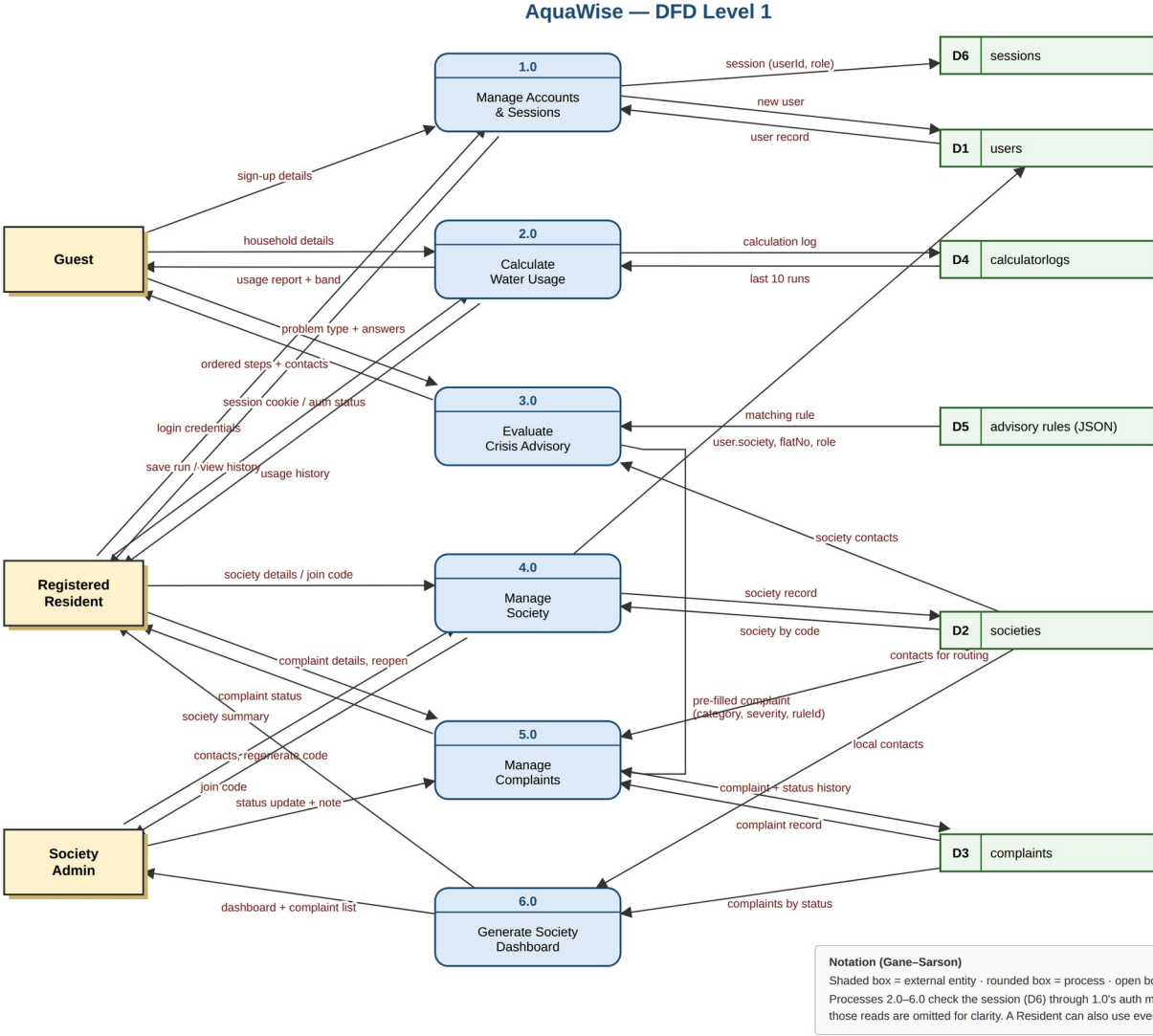


Figure 4. DFD Level 1: process 0 decomposed into 6 processes and 6 data stores (Gane-Sarson notation).

4.3 Level 1 processes and data stores

Process	What it does	Stores used
1.0 Manage Accounts & Sessions	Sign-up, login, logout; hashes passwords; creates and destroys sessions; auth middleware for all other processes.	D1 users, D6 sessions
2.0 Calculate Water Usage	Applies the usage formulas, classifies lpcd into a band, saves runs for logged-in users and returns history.	D4 calculatorlogs
3.0 Evaluate Crisis Advisory	Matches problem type + answers to one of 14 rules; returns ordered steps and contacts; can pre-fill a complaint.	D5 advisory rules, D2 societies
4.0 Manage Society	Creates societies, generates/regenerates join codes, joins members, maintains local contacts.	D2 societies, D1 users
5.0 Manage Complaints	Files, routes, updates, reopens complaints and records status history.	D3 complaints, D2 societies
6.0 Generate Society Dashboard	Counts complaints by status and lists them with local contacts.	D3 complaints, D2 societies

D1–D4 and D6 are MongoDB collections; D5 is the advisory rules JSON file (FR-A6). For readability, the session checks that processes 2.0–6.0 make through 1.0 are not drawn as separate arrows. The Level 1 flows balance with Level 0: every Level 0 input and output appears at Level 1.

5. Viva explanation of the use-case diagram

This use-case diagram shows what each kind of user can do in AquaWise. The big box is the system boundary: everything inside it is a feature of our portal, and the stick figures outside are the actors.

A Guest is anyone who has not logged in. A Guest can calculate household water usage, get crisis advice, sign up and log in. A Registered Resident is a Guest who has logged in, so the hollow-triangle arrow shows that the Resident inherits every Guest feature. On top of that, a Resident can create or join a society, file complaints, track them and view the society dashboard. The Society Admin inherits everything a Resident can do, and can also update complaint status, manage local contacts and regenerate the join code.

The System actor stands for work our server does automatically, with no person involved: classifying usage against norms, evaluating advisory rules, routing complaints and recording status history.

The dashed arrows show how use cases connect. An include arrow means a step always happens; for example, filing a complaint always includes routing it to the right person. An extend arrow means an optional step that happens only under a condition: a calculation is saved only if the user is logged in, and the 112 emergency contacts appear only when the severity is critical.

(219 words, about 1.5 minutes spoken)

Appendix A. PlantUML source — use-case diagram

Paste into plantuml.com or the PlantUML plugin for VS Code to edit. PlantUML lays the diagram out automatically, so the rendered layout differs from Figure 1; the actors, use cases and relationships are the same.

```
@startuml AquaWise_UseCase
' AquaWise — UML Use-Case Diagram (edit and re-render with PlantUML)
left to right direction
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 12
skinparam packageStyle rectangle
skinparam actorStyle awesome
skinparam usecase {
    BackgroundColor #EAF4FB
    BorderColor #1F6F9F
    ArrowColor #333333
}
skinparam rectangle {
    BorderColor #1F6F9F
    BackgroundColor #FFFFFF
}

actor "Guest" as Guest
actor "Registered\nResident" as Resident
actor "Society\nAdmin" as Admin
actor "System" as Sys <<secondary>>

Guest <|-- Resident
Resident <|-- Admin

rectangle "AquaWise Portal" as Portal {
    rectangle "Account" as Acc {
        usecase "UC09 Sign Up" as UC09
        usecase "UC10 Log In" as UC10
        usecase "UC11 Log Out" as UC11
    }
    rectangle "Water Usage Calculator" as Calc {
        usecase "UC01 Calculate\nWater Usage" as UC01
        usecase "UC02 Classify Usage\nAgainst Norms" as UC02
        usecase "UC03 Save\nCalculation" as UC03
        usecase "UC04 View Calculation\nHistory" as UC04
    }
    rectangle "Crisis Advisory Engine" as Adv {
        usecase "UC05 Get Crisis\nAdvice" as UC05
        usecase "UC06 Evaluate\nAdvisory Rules" as UC06
        usecase "UC07 Show Emergency\nContacts (112)" as UC07
        usecase "UC08 Report Issue\nto Society" as UC08
    }
    rectangle "Society Portal" as Soc {
        usecase "UC12 Join Society\nby Code" as UC12
        usecase "UC13 File\nComplaint" as UC13
        usecase "UC14 Route\nComplaint" as UC14
        usecase "UC15 Track My\nComplaints" as UC15
        usecase "UC16 Reopen\nComplaint" as UC16
        usecase "UC17 View Society\nDashboard" as UC17
        usecase "UC18 Create\nSociety" as UC18
        usecase "UC19 Generate\nJoin Code" as UC19
        usecase "UC20 Regenerate\nJoin Code" as UC20
        usecase "UC21 Update\nComplaint Status" as UC21
        usecase "UC22 Record Status\nHistory" as UC22
        usecase "UC23 Manage Local\nContacts" as UC23
    }
}

' --- Guest (inherited by Resident and Admin) ---
Guest --> UC09
Guest --> UC10
```

```

Guest --> UC01
Guest --> UC05

' --- Registered Resident ---
Resident --> UC11
Resident --> UC04
Resident --> UC12
Resident --> UC18
Resident --> UC13
Resident --> UC15
Resident --> UC17

' --- Society Admin ---
Admin --> UC20
Admin --> UC21
Admin --> UC23

' --- include (base always performs the included behaviour) ---
UC01 ..> UC02 : <<include>>
UC05 ..> UC06 : <<include>>
UC08 ..> UC13 : <<include>>
UC13 ..> UC14 : <<include>>
UC18 ..> UC19 : <<include>>
UC20 ..> UC19 : <<include>>
UC21 ..> UC22 : <<include>>

' --- extend (optional behaviour; arrow points to the base use case) ---
UC03 .> UC01 : <<extend>>\n[logged in]
UC07 .> UC05 : <<extend>>\n[severity = critical]
UC08 .> UC05 : <<extend>>\n[society member]
UC16 .> UC15 : <<extend>>\n[resolved <= 7 days]

' --- System: secondary actor performing automated steps ---
UC02 --> Sys
UC06 --> Sys
UC14 --> Sys
UC19 --> Sys
UC22 --> Sys
@enduml

```

Appendix B. Source and image files

File	Contents
images/AquaWise_UseCase.png	Use-case diagram (Figure 1)
images/AquaWise_ER.png	ER diagram (Figure 2)
images/AquaWise_DFD_Level0.png	DFD Level 0 (Figure 3)
images/AquaWise_DFD_Level1.png	DFD Level 1 (Figure 4)
source/use_case.puml	PlantUML source, use-case diagram
source/er_diagram.puml	PlantUML source, ER diagram
source/dfd_level0.dot	Graphviz source, DFD Level 0
source/AquaWise_UseCase.svg, AquaWise_DFD_Level1.svg	Vector originals of Figures 1 and 4 (editable in Inkscape or draw.io)